

LAPORAN TUGAS BESAR

ALGORITMA DAN PEMROGRAMAN 2

Aplikasi Pencatatan Konsumsi Listrik Perangkat (PowerLog)



Disusun Oleh

Kelompok:

I Wayan Yogi Dharma Putra Nananda / 108027500127

Auzan Aditya Faturrochman / 108072500004

Kelas:

IF-05-04

Dosen Pengampu:

Kharisma Monika Dian Pertiwi, S.Kom., M.Kom.

Dr. Tanzilal Mustaqim, S.Kom., M.Kom.

PROGRAM STUDI INFORMATIKA

FAKULTAS INFORMATIKA

UNIVERSITAS TELKOM

TAHUN AKADEMIK 2025 / 2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas berkat, rahmat, dan karunia-Nya sehingga penulis dapat menyelesaikan laporan Tugas Besar mata kuliah Algoritma dan Pemrograman 2 ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pengerjaan tugas besar yang merupakan salah satu syarat kelulusan mata kuliah Algoritma dan Pemrograman 2. Melalui tugas besar ini, penulis berkesempatan menerapkan konsep algoritma, pemrograman modular, dan perancangan aplikasi dalam menyelesaikan studi kasus yang diberikan.

Pada kesempatan ini penulis mengucapkan terima kasih kepada:

- Bapak/Ibu Dosen Pengampu mata kuliah Algoritma dan Pemrograman 2 atas bimbingan dan arahan yang diberikan.
- Rekan-rekan satu kelompok yang telah bekerja sama dalam penyusunan laporan dan pengembangan program.
- Pihak-pihak lain yang turut mendukung penyelesaian tugas besar ini.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca.

Surabaya, 15 Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I	
PENDAHULUAN	1
1.1 Ringkasan Tugas Besar	1
1.2 Tujuan	1
BAB II	1
DESKRIPSI TUGAS BESAR	2
2.1 Gambaran Umum Aplikasi	2
2.2 Fitur-Fitur Aplikasi	2
1. CRUD (Create, Read, Update, Delete)	2
2. Searching	2
3. Statistic Summary	3
2.3 Flowchart Fitur	3
• Flowchart Tampilan login dan main menu	3
• Flowchart Tambah perangkat	4
• Flowchart Ubah detail Perangkat	5
• Flowchart Hapus perangkat	6
• Flowchart cari berdasarkan nama	7
• Flowchart cari berdasarkan Ruangan	8
• Flowchart Statistik Perangkat	9
BAB III	
PENJELASAN PROGRAM	11
3.1 Struktur Program Modular	11
A. Data Structure	11
B. Fungsi Helper (Utility)	11
C. Algoritma (Sorting & Searching)	11
D. Modul Fitur (Logic & CRUD)	12
E. Modul Tampilan (Menu)	12
F. Fungsi Utama (Main)	13
3.2 Pseudocode	13
1. Data Perangkat	13
2. CRUD Fitur (Tambah Perangkat)	13
3. CRUD Fitur (Ubah Perangkat)	14
4. CRUD Fitur (Hapus Perangkat)	16
5. CRUD Fitur (Baca Perangkat)	17
6. Selection Sort (Tampilkan Berdasarkan Konsumsi Energi Tertinggi)	18
7. Insertion Sort (Tampilkan Berdasarkan Abjad Perangkat)	18
8. Sequential Searching (Tampilkan Berdasarkan Nama Perangkat)	19
9. Binary Search (Tampilkan Berdasarkan Ruangan)	19
10. Sort Ruangan (Mengurutkan Data Sebelum Masuk Di Binary Search)	21

11. Statistik Perangkat (Total Konsumsi Harian)	21
12. Statistik Perangkat (Daftar Perangkat Konsumsi Listrik Tertinggi)	22
13. Statistik Perangkat (Daftar Perangkat Paling Boros)	22
14. Fungsi Helper (Validasi, cekInputKosong, hitungKonsumsi, cekAdaPerangkat, tampilkanPerangkat)	22
3.3 Penjelasan Kode Program	24
1. Struct dan Array	24
2. Fungsi Helper (Validasi)	24
3. Fungsi Helper (cekInputKosong)	25
4. Fungsi Helper (Konversi kWh)	25
5. Fungsi Helper (Tampilkan perangkat)	26
6. Fungsi Helper (Clear screen)	26
7. Fungsi Helper (Pause)	27
8. Fungsi Helper (cekAdaPerangkat)	27
9. Selection Sort (Daya perangkat)	28
10. Insertion Sort (Abjad perangkat)	28
11. Selection Sort (Konsumsi daya tertinggi perangkat)	29
12. Binary Search (Mencari berdasarkan nama ruangan)	30
13. Sequential Search (Mencari berdasarkan nama perangkat)	32
BAB IV	
TANTANGAN DAN SOLUSI	33
1. Validasi input di sistem pilih menu	33
2. Bug saat user menekan “Enter” saat memasuki sub menu	33
3. Duplikasi data saat proses input dan mengubah data	33
4. Proses input untuk dayaW	34
5. Proses membuat algoritma Sorting dan Searching	34
BAB V	
KESIMPULAN DAN REKOMENDASI	35
5.1 Kesimpulan	35
5.2 Rekomendasi	35
TAUTAN GITHUB	36
REFERENSI	37

BAB I

PENDAHULUAN

1.1 Ringkasan Tugas Besar

Penggunaan perangkat elektronik di rumah maupun tempat kerja terus meningkat seiring perkembangan teknologi. Namun, banyak pengguna yang belum mengetahui seberapa besar konsumsi listrik dari setiap perangkat yang digunakan setiap hari. Hal ini dapat menyebabkan pemborosan energi dan meningkatnya biaya listrik.

Oleh karena itu, dibuat aplikasi pencatatan konsumsi listrik perangkat (**PowerLog**) untuk membantu pengguna mencatat data perangkat elektronik, menghitung konsumsi energi harian, serta menampilkan informasi perangkat dengan konsumsi listrik tertinggi. Dengan adanya aplikasi ini, pengguna dapat memantau penggunaan listrik secara lebih mudah dan mengambil keputusan yang lebih baik dalam mengelola konsumsi energi.

1.2 Tujuan

- **Menerapkan konsep algoritma dan pemrograman** yang telah dipelajari selama perkuliahan, seperti penggunaan struktur data, fungsi, pencarian (*searching*), pengurutan (*sorting*), serta pengolahan data menggunakan bahasa pemrograman Go.
- **Mengembangkan aplikasi PowerLog Apps** yang dapat membantu pengguna dalam mengelola data perangkat elektronik, menghitung konsumsi energi listrik, dan menampilkan informasi penggunaan listrik secara sederhana dan mudah dipahami.
- **Melatih kemampuan analisis, perancangan, dan implementasi program** mulai dari mengidentifikasi kebutuhan sistem, merancang alur program, hingga mengimplementasikan fitur-fitur yang sesuai dengan studi kasus yang dipilih.
- **Meningkatkan kemampuan pemecahan masalah** melalui pembuatan solusi berbasis program untuk membantu pengguna memantau dan mengelola konsumsi energi listrik dari berbagai perangkat elektronik.

BAB II

DESKRIPSI TUGAS BESAR

2.1 Gambaran Umum Aplikasi

- Aplikasi Pencatatan Konsumsi Listrik Perangkat (**PowerLog**)
- Powerlog adalah aplikasi yang dibuat untuk memantau dan mencatat penggunaan daya listrik pada berbagai alat elektronik rumah tangga. Data utama yang digunakan adalah data perangkat, daya (watt), dan durasi penggunaan.
- Pengguna aplikasi adalah pemilik rumah tangga yang ingin mengontrol efisiensi energi
- Pada proyek kali ini kita menggunakan bahasa pemrograman GO

2.2 Fitur-Fitur Aplikasi

Aplikasi PowerLog memiliki beberapa fitur yang dapat memudahkan pengguna dalam mencatat maupun analisa konsumsi perangkat elektronik mereka. Antara lain:

1. CRUD (*Create, Read, Update, Delete*)

Aplikasi kami dapat mencatat perangkat, lokasi perangkat, besaran daya (dalam watt), dan berapa lama (dalam jam) perangkat tersebut terhubung ke sumber listrik. Aplikasi kami juga bisa untuk membaca perangkat berdasarkan konsumsi listrik yang tertinggi maupun berdasarkan abjad perangkat (A - Z) secara akurat. Pengguna juga dapat melakukan perubahan pada data perangkat yang sudah ada dan langsung dihitung besaran kWh (dari data watt elektronik) untuk mendapatkan statistik yang lebih akurat. Lalu pengguna juga bisa menghapus data perangkat yang ada untuk memudahkan management file.

2. Searching

Pengguna juga dapat melakukan pencarian perangkat berdasarkan ruangan dimana perangkat itu diletakan maupun pencarian berdasarkan nama perangkat itu sendiri.

3. Statistic Summary

Pengguna tentunya dapat melihat statistik perangkat elektroniknya berdasarkan perangkat mana yang memiliki konsumsi daya terboros, total konsumsi seluruh perangkat dalam sehari, maupun daftar perangkat yang memiliki konsumsi listrik yang tinggi.

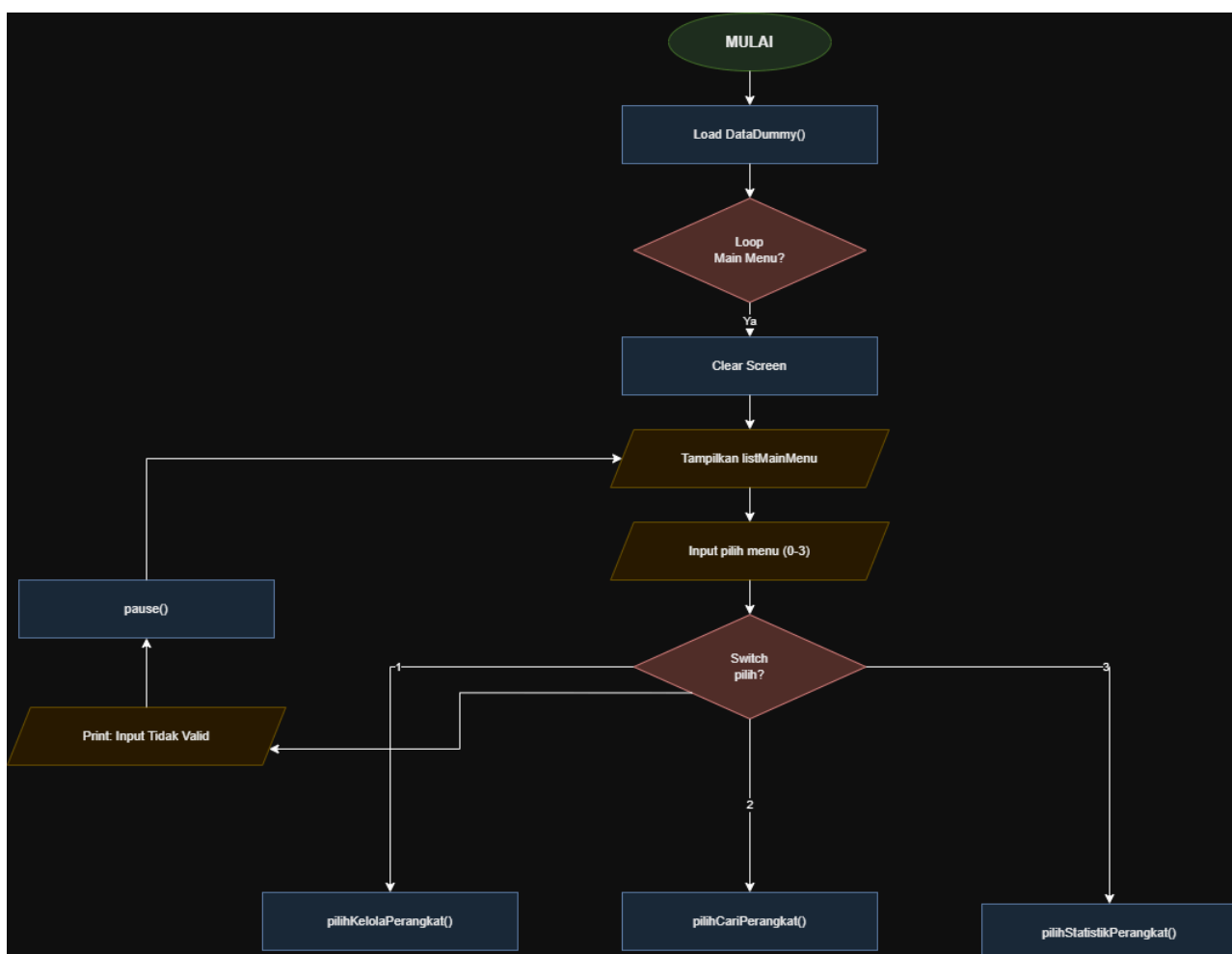
Apps kami juga telah memiliki sistem validasi yang cukup akurat dalam mengatasi error input dari user.

2.3 Flowchart Fitur

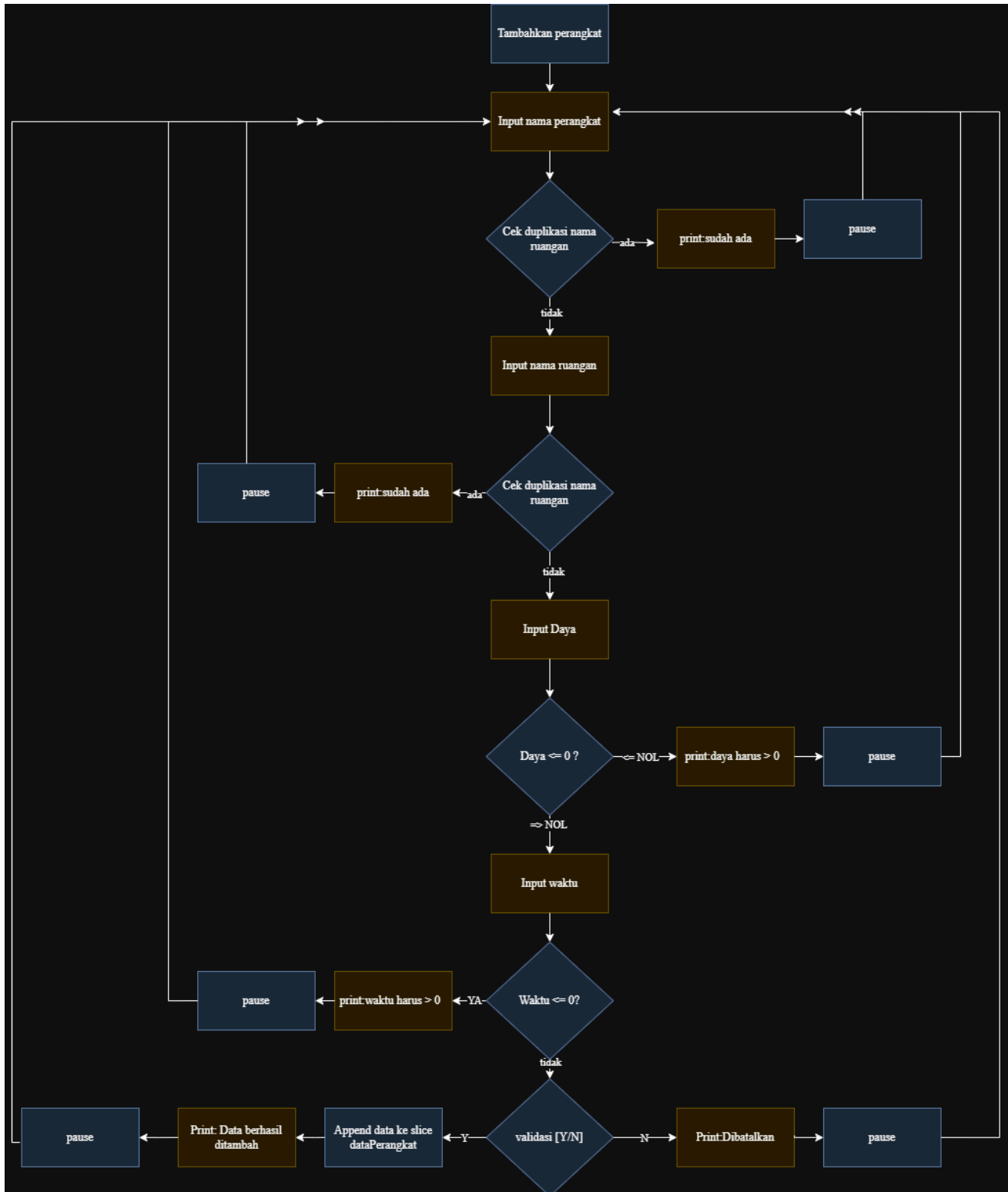
Sisipkan flowchart untuk menggambarkan alur proses dari fitur-fitur utama aplikasi. Setiap flowchart sebaiknya diberi judul dan penjelasan singkat.

Contoh:

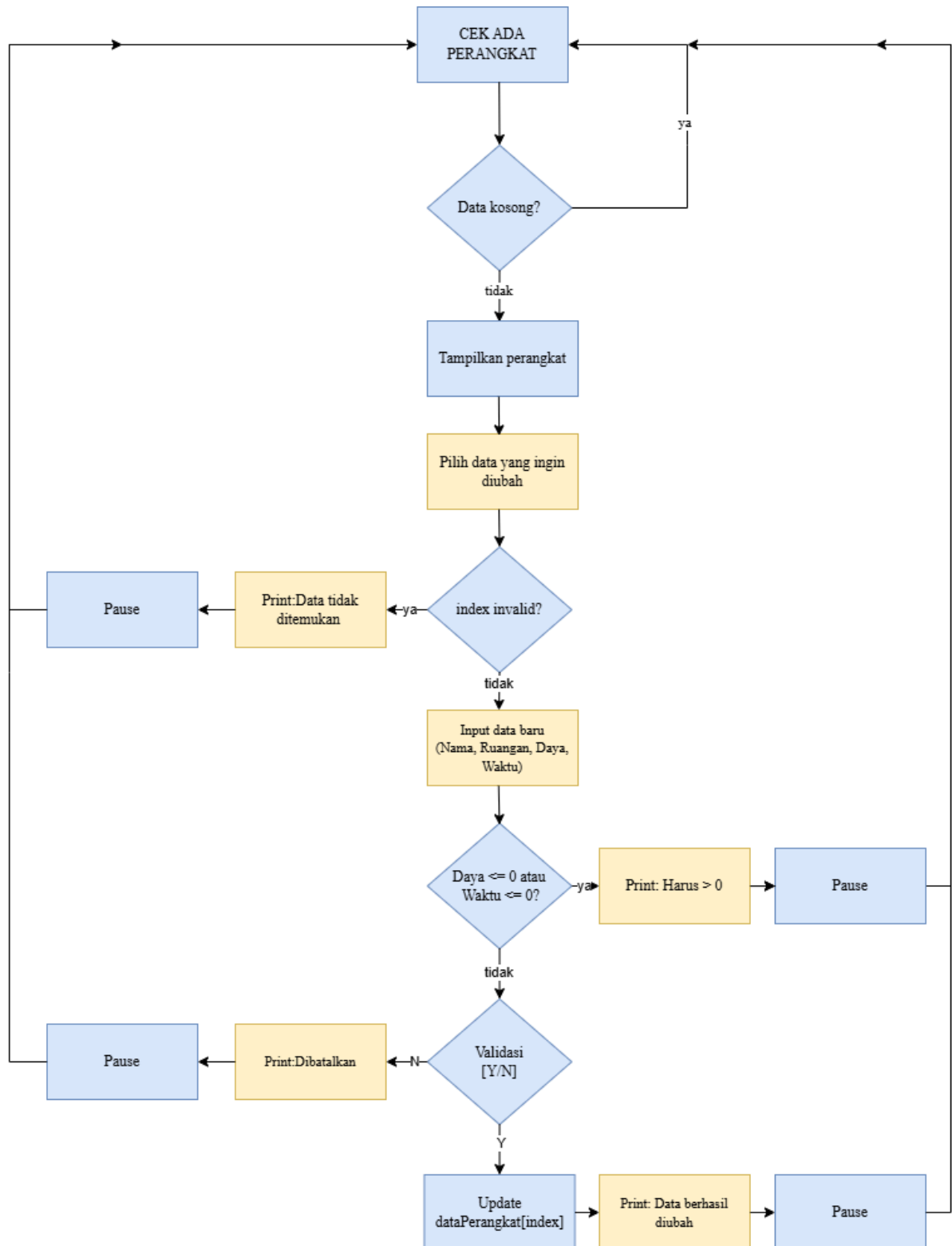
- Flowchart Tampilan login dan main menu



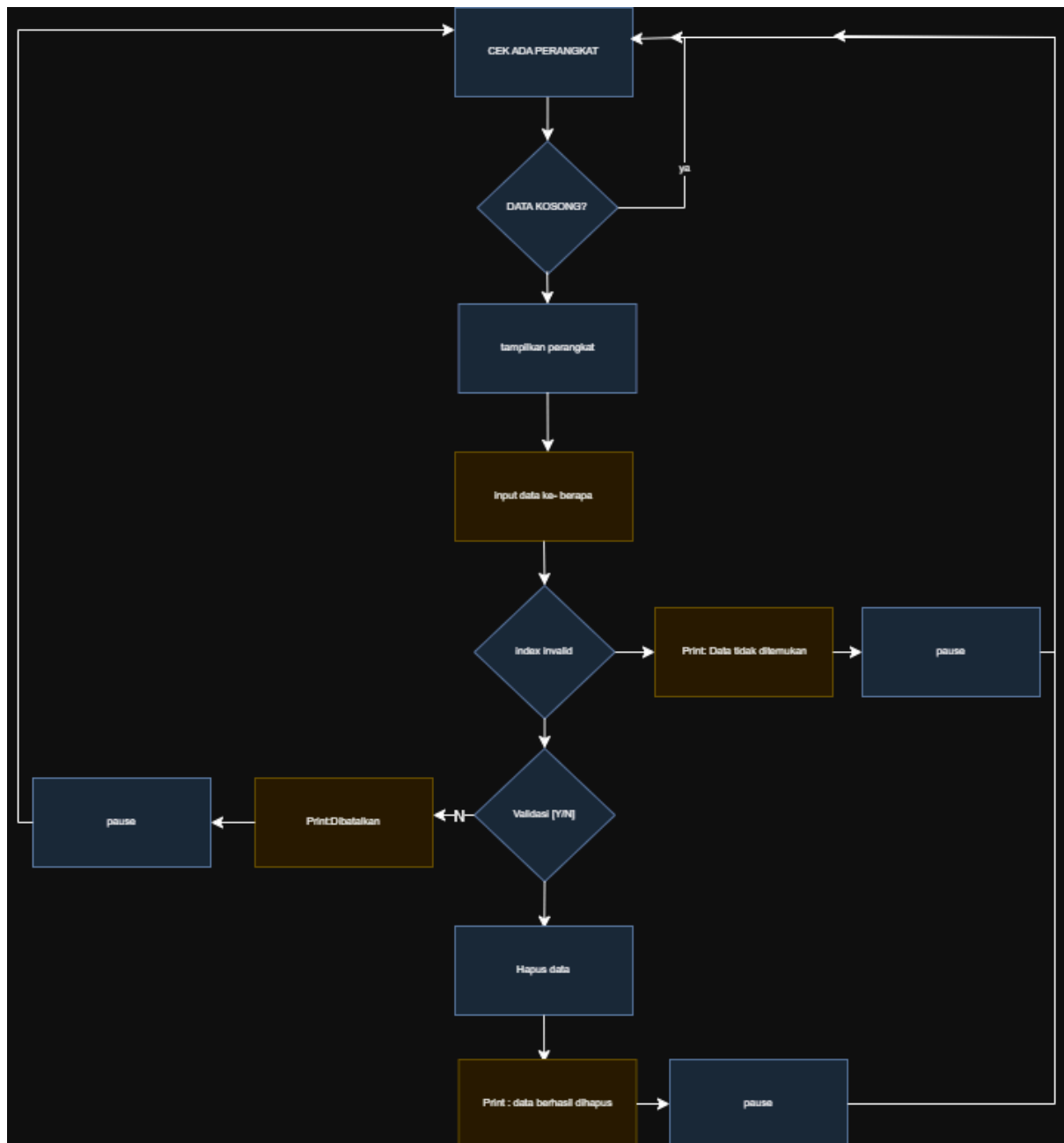
- Flowchart Tambah perangkat



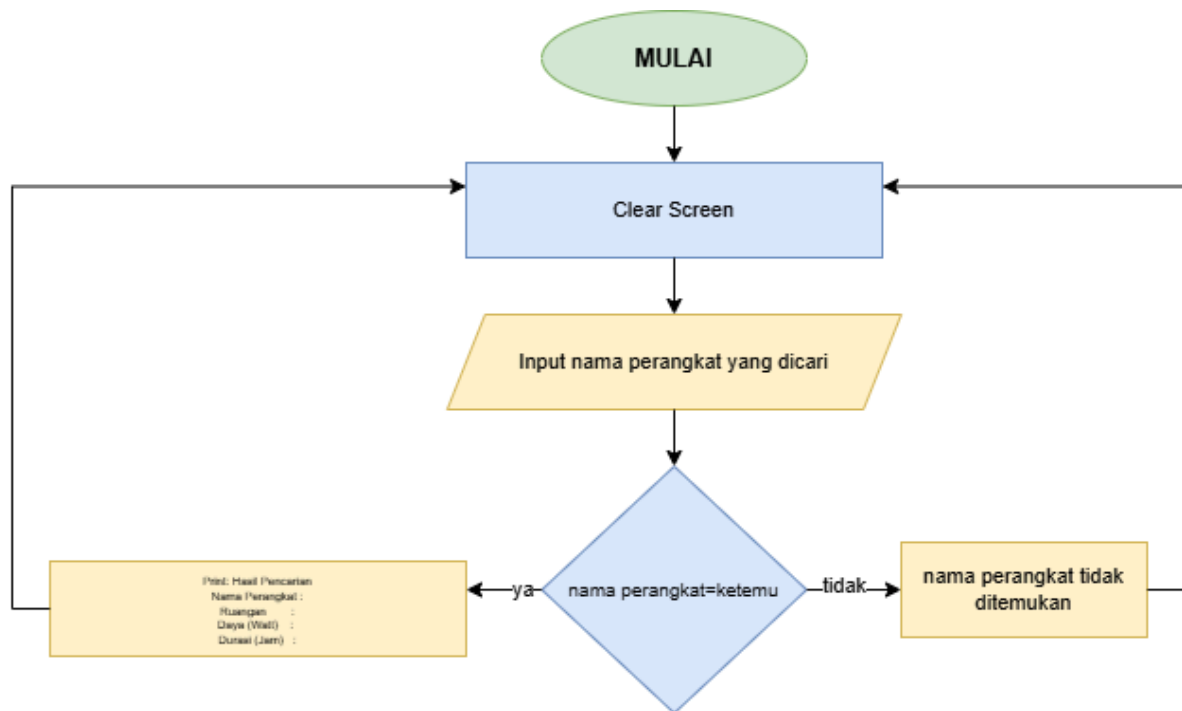
- Flowchart Ubah detail Perangkat



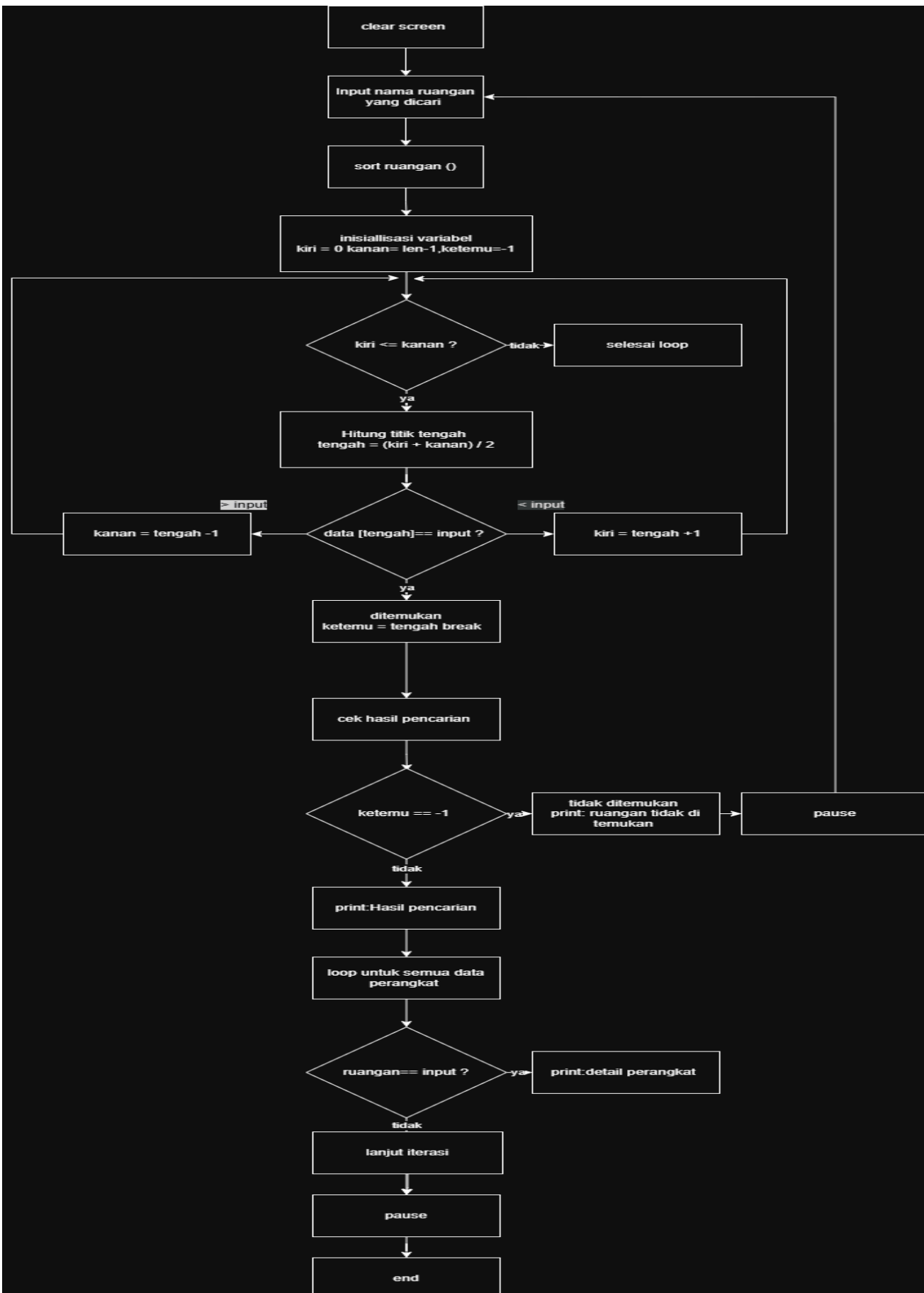
- Flowchart Hapus perangkat



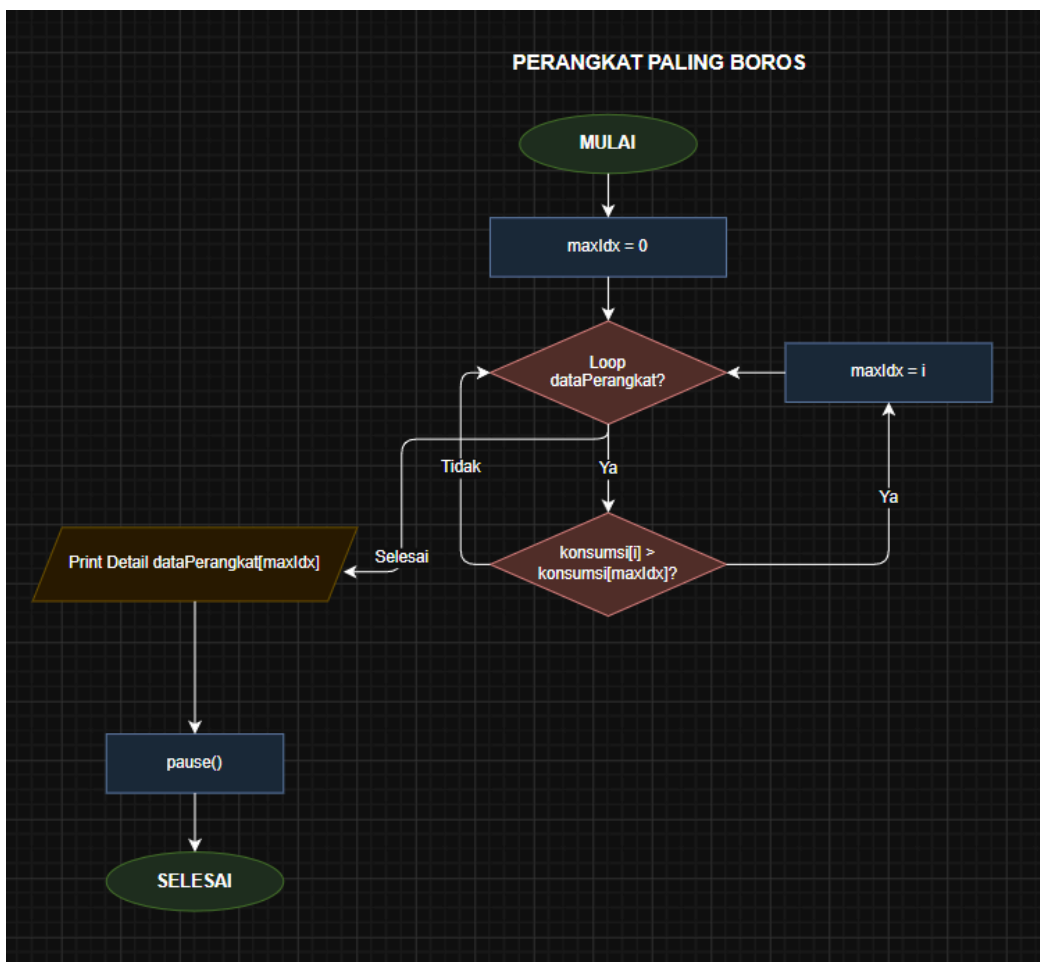
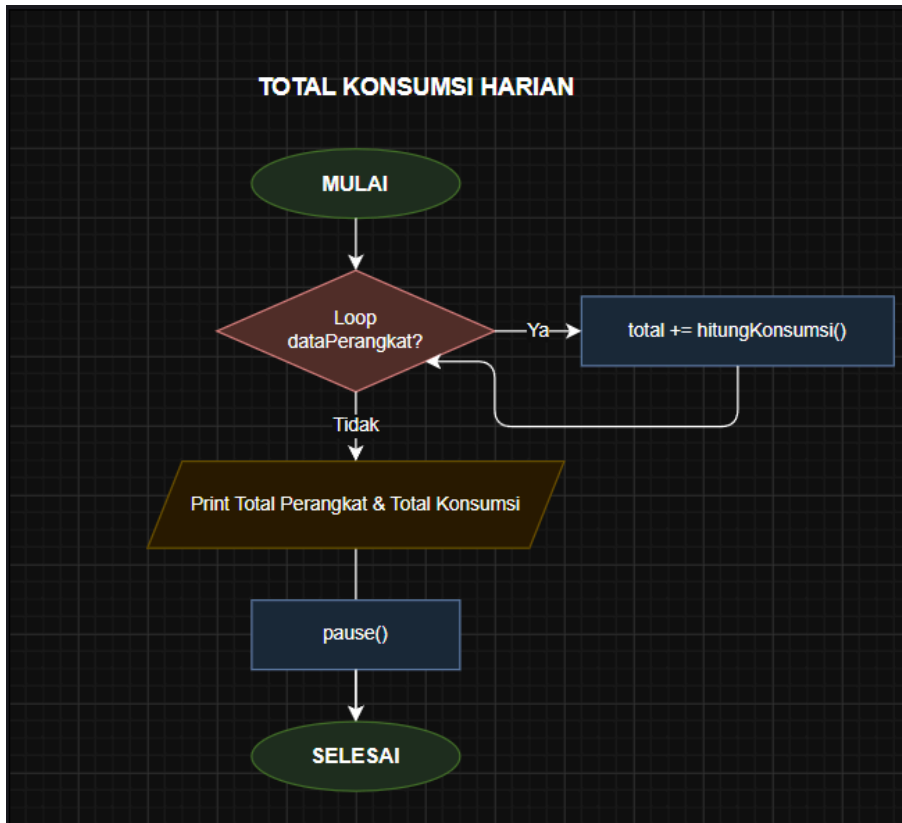
- Flowchart cari berdasarkan nama



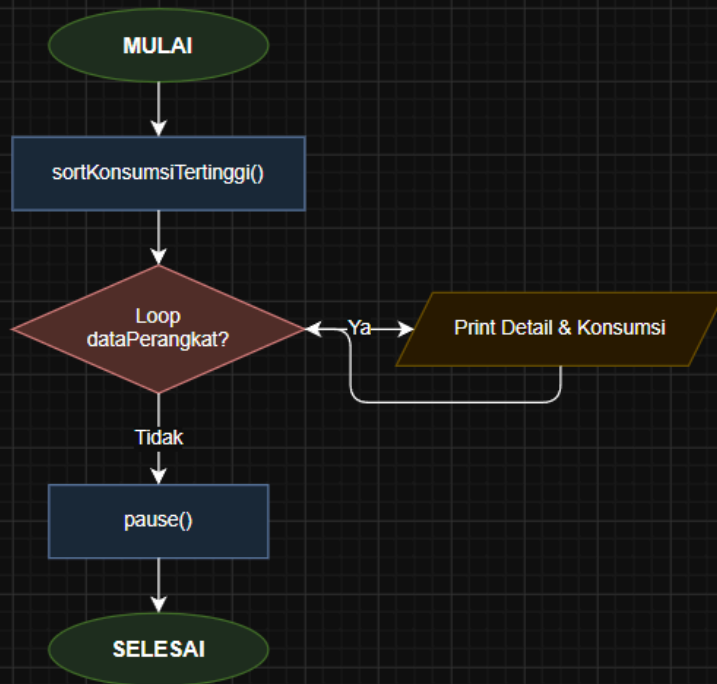
- Flowchart cari berdasarkan Ruangan



- Flowchart Statistik Perangkat



DAFTAR KONSUMSI TERTINGGI



BAB III

PENJELASAN PROGRAM

3.1 Struktur Program Modular

A. Data Structure

type Perangkat struct: Untuk menyimpan atribut namaPerangkat, ruangan, dayaW, dan waktu sebagai data yang dibutuhkan dalam sistem PoweLog.

B. Fungsi Helper (Utility)

Fungsi-fungsi ini berfungsi sebagai fungsi yang bisa dipakai berulang oleh sistem sehingga lebih efisien dalam pengembangannya:

- validasi(): Berfungsi untuk menahan user sebelum mengeksekusi perintah utama disistem.
- cekInputKosong(): Berfungsi untuk mencegah user memasukkan input kosong pada data string.
- hitungKonsumsi(): Berfungsi untuk perhitungan matematis untuk melakukan konversi Watt menjadi kWh.
- clearScreen(): Berfungsi untuk melakukan eksekusi command pembersihan terminal pada CLI (windows atau mac).
- pause(): Berfungsi untuk menahan alur program agar output dapat dibaca sebelum mengeksekusi fungsi clearScreen().
- cekAdaPerangkat(): Memeriksa apakah ada data didalam slice untuk memastikan program tidak running apabila data dalam slice kosong (alias tidak ada).
- DataDummy(): Berfungsi untuk memberikan contoh data kedalam sistem agar aplikasi bisa berjalan.

C. Algoritma (Sorting & Searching)

Modul ini berfungsi untuk mengolah data dengan menggunakan algoritma tertentu:

- selectionSortPerangkat(): Mengurutkan data perangkat berdasarkan konsumsi daya terbesar ke terkecil.

- `insertionSortPerangkat()`: Mengurutkan data perangkat berdasarkan nama perangkat secara alfabet (A-Z).
- `sortRuangan()`: Mengurutkan data berdasarkan nama ruangan menggunakan metode `selection sort`.
- `cariNamaPerangkat()`: Melakukan pencarian perangkat berdasarkan nama dengan metode `looping` (pencarian sekuensial).
- `cariRuangan()`: Melakukan pencarian berdasarkan nama ruangan dengan metode `binary search` setelah data diurutkan terlebih dahulu.

D. Modul Fitur (Logic & CRUD)

Fungsi yang menjalankan inti program aplikasi PowerLog:

- `tampilkanPerangkat()`: Menampilkan seluruh data yang ada di dalam slice.
- `totalKonsumsiHarian()`, `daftarKonsumsiTertinggi()`, `perangkatPalingBoros()`: Fungsi yang melakukan perhitungan dan menampilkan statistik konsumsi energi.
- `tambahPerangkat()`: Menambah data baru ke slice.
- `ubahPerangkat()`: Memperbarui data perangkat berdasarkan indeks yang dipilih.
- `hapusPerangkat()`: Menghapus data dari slice berdasarkan indeks yang dipilih oleh user.

E. Modul Tampilan (Menu)

Fungsi yang berperan sebagai *interface* CLI untuk mengarahkan alur program:

- `listMainMenu()`, `MenuKelolaPerangkat()`, `MenuCariPerangkat()`, `MenuStatistikPerangkat()`, `menuUrutkanPerangkat()`: Berfungsi untuk menampilkan daftar opsi menu kepada user.
- `inputMainMenu()` & fungsi pilih (seperti: `pilihKelolaPerangkat()`, `pilihCariPerangkat()`, `pilihUrutkanPerangkat()`, `pilihStatistikPerangkat()`): Berfungsi untuk kontrol alur dengan `switch-case` untuk mengeksekusi modul fitur berdasarkan input angka dari pengguna.

F. Fungsi Utama (Main)

main(): Fungsi utama yang memanggil DataDummy() untuk inisialisasi data sampel dan menjalankan inputMainMenu() untuk memulai program.

3.2 Pseudocode

1. Data Perangkat

```
kamus global
    type Perangkat : record <
        namaPerangkat : string,
        ruangan        : string,
        dayaW          : real,
        waktu           : real
    >

    dataPerangkat : array of Perangkat
```

2. CRUD Fitur (Tambah Perangkat)

```
procedure tambahPerangkat()
kamus
    tambah : Perangkat
    i : integer
algoritma
    clearScreen()
    output("==== TAMBAH PERANGKAT =====\n")

    output("Masukkan Nama Perangkat:")
    input(tambah.namaPerangkat)
    if not cekInputKosong(tambah.namaPerangkat) then
        output("Input tidak boleh kosong!")
        pause()
        return
    endif

    output("Masukkan Nama Ruangan: ")
    input(tambah.ruangan)
    if not cekInputKosong(tambah.ruangan) then
        output("Input tidak boleh kosong!")
        pause()
        return
    endif

    for i = 0 to length(dataPerangkat) - 1 do
        if tambah.namaPerangkat == dataPerangkat[i].namaPerangkat
then
```

```

        if tambah.ruangan == dataPerangkat[i].ruangan then
            output("Nama perangkat sudah ada diruangan yang
sama")
            pause()
            return
        endif
    endif
endfor

output("Masukkan Daya Perangkat (watt): ")
input(tambah.dayaw)
if tambah.dayaw <= 0 then
    output("Daya harus lebih dari 0")
    pause()
    return
endif

output("Masukkan Lama Pemakaian (jam): ")
input(tambah.waktu)
if tambah.waktu <= 0 then
    output("Waktu harus lebih dari 0")
    pause()
    return
endif

if not validasi() then
    clearScreen()
    output("Input dibatalkan")
    pause()
    return
endif

clearScreen()
append(dataPerangkat, tambah)

output("==== INPUT TELAH DITAMBAHKAN =====\n")
output("==== DETAIL INPUT =====\n")
output("Nama Perangkat : ", tambah.namaPerangkat)
output("Ruangan      : ", tambah.ruangan)
output("Daya (watt)    : ", tambah.dayaw)
output("Waktu (jam)    : ", tambah.waktu)

pause()
endprocedure

```

3. CRUD Fitur (Ubah Perangkat)

```

procedure ubahPerangkat()
kamus
    pilihUbah, idx, i : integer

```

```

    namaBaru, ruanganBaru : string
    dayaBaru, waktuBaru : real
algoritma
    clearScreen()
    if cekAdaPerangkat() then
        return
    endif

    tampilkanPerangkat()
    output("==== UBAH DETAIL PERANGKAT =====")
    output("Pilih data ke- berapa yang ingin diubah: ")
    input(pilihUbah)

    if pilihUbah == error then
        output("Input harus berupa angka!")
        pause()
        return
    endif

    idx = pilihUbah - 1
    if idx < 0 or idx >= length(dataPerangkat) then
        output("Data tidak ditemukan!")
        pause()
        return
    endif

    output("\n==== DATA BARU =====")
    output("Nama Perangkat : ")
    input(namaBaru)
    if not cekInputKosong(namaBaru) then
        output("Input tidak boleh kosong!")
        pause()
        return
    endif

    output("Ruangan : ")
    input(ruanganBaru)
    if not cekInputKosong(ruanganBaru) then
        output("Input tidak boleh kosong!")
        pause()
        return
    endif

    for i = 0 to length(dataPerangkat) - 1 do
        if namaBaru == dataPerangkat[i].namaPerangkat then
            if ruanganBaru == dataPerangkat[i].ruangan then
                output("Nama perangkat sudah ada diruangan yang
sama")
                pause()
                return
            endif
        endif
    endif
endif

```

```

endfor

output("Daya (Watt) : ")
input(dayaBaru)
if dayaBaru <= 0 then
    output("Daya harus lebih dari 0")
    pause()
    return
endif

output("Lama Pemakaian (Jam) : ")
input(waktuBaru)
if waktuBaru <= 0 then
    output("Waktu harus lebih dari 0")
    pause()
    return
endif

if not validasi() then
    output("Perubahan dibatalkan")
    pause()
    return
endif

dataPerangkat[idx].namaPerangkat = namaBaru
dataPerangkat[idx].ruangan = ruanganBaru
dataPerangkat[idx].dayaW = dayaBaru
dataPerangkat[idx].waktu = waktuBaru

output("\nData berhasil diubah!")
pause()
endprocedure

```

4. CRUD Fitur (Hapus Perangkat)

```

procedure hapusPerangkat()
kamus
    pilihHapus, idx : integer
algoritma
    clearScreen()
    if cekAdaPerangkat() then
        return
    endif

    tampilkanPerangkat()
    output("==== HAPUS PERANGKAT =====")
    output("Pilih data ke- berapa yang ingin dihapus : ")
    input(pilihHapus)

    if pilihHapus == error then

```

```

        output("Masukkan nomor yang valid!")
        pause()
        return
    endif

    idx = pilihHapus - 1
    if idx < 0 or idx >= length(dataPerangkat) then
        output("Data tidak ditemukan!")
        pause()
        return
    endif

    if not validasi() then
        output("Penghapusan dibatalkan")
        pause()
        return
    endif

    remove(dataPerangkat, idx)
    clearScreen()
    output("\nData perangkat berhasil dihapus!")
    pause()
endprocedure

```

5. CRUD Fitur (Baca Perangkat)

```

procedure tampilkanPerangkat()
kamus
    i : integer
algoritma
    // Melakukan cek apabila isi array/slice = 0
    if cekAdaPerangkat() == true then
        return
    endif

    output("===== DATA PERANGKAT =====")

    // Looping untuk membaca (read) dan mencetak data
    for i = 0 to length(dataPerangkat) - 1 do
        output("Data ke-", i+1)
        output("Nama Perangkat :", dataPerangkat[i].namaPerangkat)
        output("Ruangan      :", dataPerangkat[i].ruangan)
        output("Daya (Watt)   :", dataPerangkat[i].dayaW)
        output("Durasi (Jam)   :", dataPerangkat[i].waktu)
        output("Konsumsi      :",
hitungKonsumsi(dataPerangkat[i]), "kWh")
        output("-----")
    endfor
endprocedure

```

6. Selection Sort (Tampilkan Berdasarkan Konsumsi Energi Tertinggi)

```
procedure selectionSortPerangkat()
kamus
    n, i, j, maxIdx : integer
    temp : Perangkat
algoritma
    n = length(dataPerangkat)

    for i = 0 to n - 2 do
        maxIdx = i

        for j = i + 1 to n - 1 do
            // Membandingkan daya dari yang paling besar
            (Descending)
            if dataPerangkat[j].dayaW > dataPerangkat[maxIdx].dayaW
then
                maxIdx = j
            endif
        endfor

        // Proses pertukaran data (Swap)
        temp = dataPerangkat[i]
        dataPerangkat[i] = dataPerangkat[maxIdx]
        dataPerangkat[maxIdx] = temp
    endfor
endprocedure
```

7. Insertion Sort (Tampilkan Berdasarkan Abjad Perangkat)

```
procedure insertionSortPerangkat()
kamus
    n, i, j : integer
    key : Perangkat
algoritma
    n = length(dataPerangkat)

    for i = 1 to n - 1 do
        key = dataPerangkat[i]
        j = i - 1

        // Membandingkan dan menggeser string untuk urutan A-Z
        (Ascending)
        while j >= 0 and dataPerangkat[j].namaPerangkat >
key.namaPerangkat do
            dataPerangkat[j+1] = dataPerangkat[j]
            j = j - 1
        endwhile
        dataPerangkat[j+1] = key
    endfor
```

```

        endwhile
        dataPerangkat[j+1] = key
    endfor
endprocedure

```

8. Sequential Searching (Tampilkan Berdasarkan Nama Perangkat)

```

procedure cariNamaPerangkat()
kamus
    cariberdnama : string
    ketemu : boolean
    i : integer
algoritma
    ketemu = false
    clearScreen()

    output("Masukkan nama perangkat yang dicari : ")
    input(cariberdnama)

    if not cekInputKosong(cariberdnama) then
        output("Input tidak boleh kosong!")
        pause()
        return
    endif

    output("\n==== HASIL PENCARIAN =====")
    for i = 0 to length(dataPerangkat) - 1 do
        if dataPerangkat[i].namaPerangkat == cariberdnama then
            output("Nama Perangkat :",
dataPerangkat[i].namaPerangkat)
            output("Ruangan          :", dataPerangkat[i].ruangan)
            output("Daya (Watt)       :", dataPerangkat[i].dayaW)
            output("Durasi (Jam)      :", dataPerangkat[i].waktu)
            output("-----")
            ketemu = true
        endif
    endfor

    if not ketemu then
        output("Perangkat tidak ditemukan!")
    endif
    pause()
endprocedure

```

9. Binary Search (Tampilkan Berdasarkan Ruangan)

```

procedure cariRuangan()
kamus

```

```

    cariberdruangan : string
    kiri, kanan, tengah, ketemu, i : integer
algoritma
    clearScreen()
    output("Masukkan nama ruangan yang dicari : ")
    input(cariberdruangan)

    if not cekInputKosong(cariberdruangan) then
        output("Input tidak boleh kosong!")
        pause()
        return
    endif

    sortRuangan()

    kiri = 0
    kanan = length(dataPerangkat) - 1
    ketemu = -1

    while kiri <= kanan do
        tengah = (kiri + kanan) div 2
        if dataPerangkat[tengah].ruangan == cariberdruangan then
            ketemu = tengah
            break
        else if dataPerangkat[tengah].ruangan < cariberdruangan
then
            kiri = tengah + 1
        else
            kanan = tengah - 1
        endif
    endwhile

    if ketemu == -1 then
        output("\nRuangan tidak ditemukan!")
        pause()
        return
    endif

    output("\n===== HASIL PENCARIAN =====")
    for i = 0 to length(dataPerangkat) - 1 do
        if dataPerangkat[i].ruangan == cariberdruangan then
            output("-----")
            output("Nama Perangkat :",
dataPerangkat[i].namaPerangkat)
            output("Ruangan          :", dataPerangkat[i].ruangan)
            output("Daya (Watt)       :", dataPerangkat[i].dayaW)
            output("Durasi (Jam)      :", dataPerangkat[i].waktu)
        endif
    endfor
    pause()
endprocedure

```


10. Sort Ruangan (Mengurutkan Data Sebelum Masuk Di Binary Search)

```
procedure sortRuangan()
kamus
    n, i, j, minIdx : integer
    temp : Perangkat
algoritma
    n = length(dataPerangkat)
    for i = 0 to n - 2 do
        minIdx = i
        for j = i + 1 to n - 1 do
            if dataPerangkat[j].ruangan <
dataPerangkat[minIdx].ruangan then
                minIdx = j
            endif
        endfor

        temp = dataPerangkat[i]
        dataPerangkat[i] = dataPerangkat[minIdx]
        dataPerangkat[minIdx] = temp
    endfor
endprocedure
```

11. Statistik Perangkat (Total Konsumsi Harian)

```
procedure totalKonsumsiHarian()
kamus
    total, konsumsi : real
    banyakPerangkat, i : integer
algoritma
    clearScreen()
    total = 0
    banyakPerangkat = 0

    output("==== TOTAL KONSUMSI ENERGI HARIAN =====")
    for i = 0 to length(dataPerangkat) - 1 do
        konsumsi = hitungKonsumsi(dataPerangkat[i])
        total = total + konsumsi
        banyakPerangkat = banyakPerangkat + 1
    endfor

    tampilkanPerangkat()
    output("Total Perangkat:", banyakPerangkat, "Perangkat")
    output("Total Konsumsi Harian:", total, "kWh")
    pause()
endprocedure
```

12. Statistik Perangkat (Daftar Perangkat Konsumsi Listrik Tertinggi)

```
procedure daftarKonsumsiTertinggi()
kamus
    algoritma
        clearScreen()
        sortKonsumsiTertinggi() // Memanggil fungsi sort berdasarkan
konsumsi (mirip selection sort daya)

        output("==== DAFTAR PERANGKAT KONSUMSI TERTINGGI =====")
        tampilkanPerangkat()
        pause()
endprocedure
```

13. Statistik Perangkat (Daftar Perangkat Paling Boros)

```
procedure perangkatPalingBoros()
kamus
    maxIdx, i : integer
algoritma
    clearScreen()
    maxIdx = 0

    for i = 1 to length(dataPerangkat) - 1 do
        if hitungKonsumsi(dataPerangkat[i]) >
hitungKonsumsi(dataPerangkat[maxIdx]) then
            maxIdx = i
        endif
    endfor

    output("==== PERANGKAT PALING BOROS LISTRIK =====")
    output("Nama Perangkat :", dataPerangkat[maxIdx].namaPerangkat)
    output("Ruangan          :", dataPerangkat[maxIdx].ruangan)
    output("Durasi (Jam)       :", dataPerangkat[maxIdx].waktu)
    output("Konsumsi           :",
hitungKonsumsi(dataPerangkat[maxIdx]), "kWh")
    pause()
endprocedure
```

14. Fungsi Helper (Validasi, cekInputKosong, hitungKonsumsi, cekAdaPerangkat, tampilkanPerangkat)

```
function validasi() -> boolean
kamus
    confirm : string
```

```

    algoritma
        while true do
            output("Apakah anda yakin? [Y/y or N/n]: ")
            input(confirm)
            if confirm == "Y" or confirm == "y" then
                return true
            else if confirm == "N" or confirm == "n" then
                return false
            else
                output("Input tidak valid, masukkan antara [Y/y or
N/n]!")
            endif
        endwhile
    endfunction

function cekInputKosong(inputStr : string) -> boolean
kamus
algoritma
    if inputStr == "" then
        return false
    else
        return true
    endif
endfunction

function hitungKonsumsi(perangkat : Perangkat) -> real
kamus
algoritma
    return (perangkat.dayaw * perangkat.waktu) / 1000
endfunction

function cekAdaPerangkat() -> boolean
kamus
algoritma
    if length(dataPerangkat) == 0 then
        output("Belum ada data perangkat\n")
        output("Silahkan tambahkan perangkat terlebih dahulu...")
        pause()
        return true
    endif
    return false
endfunction

procedure tampilkanPerangkat()
kamus
    i : integer
algoritma
    if cekAdaPerangkat() == true then
        return
    endif

    output("===== DATA PERANGKAT =====")

```

```

    for i = 0 to length(dataPerangkat) - 1 do
        output("Data ke-", i+1)
        output("Nama Perangkat :", dataPerangkat[i].namaPerangkat)
        output("Ruangan      :", dataPerangkat[i].ruangan)
        output("Daya (Watt)   :", dataPerangkat[i].dayaW)
        output("Durasi (Jam)   :", dataPerangkat[i].waktu)
        output("Konsumsi      :",
hitungKonsumsi(dataPerangkat[i]), "kWh")
        output("-----")
    endfor
endprocedure

```

3.3 Penjelasan Kode Program

1. Struct dan Array

```

1 // STRUCT PERANGKAT UNTUK SIMPAN DATA PERANGKAT
2 type Perangkat struct {
3     namaPerangkat string
4     ruangan        string
5     dayaW          float64
6     waktu          float64
7 }
8
9 // SLICE UNTUK dataPerangkat DARI STRUCT PERANGKAT
10 var dataPerangkat []Perangkat

```

Pada bagian ini, kami menginisiasi variable bernama “Perangkat” sebagai struct untuk menyimpan data - data yang digunakan oleh apps kami (seperti: namaPerangkat, ruangan, dayaW, dan waktu). Dan, kami menggunakan slice [] dengan variable dataPerangkat untuk menyimpan data array yang lebih fleksibel.

2. Fungsi Helper (Validasi)

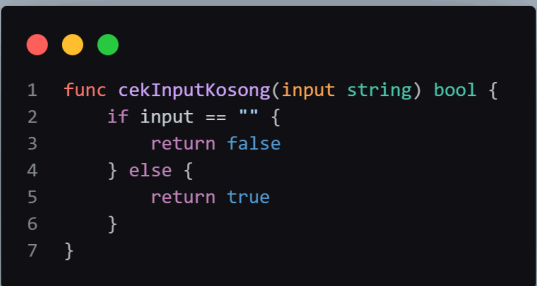
```

1 func validasi() bool {
2     var confirm string
3
4     for {
5         fmt.Print("Apakah anda yakin? [Y/y or N/n]: ")
6         fmt.Scanln(&confirm)
7         if confirm == "Y" || confirm == "y" {
8             return true
9         } else if confirm == "N" || confirm == "n" {
10            return false
11        } else {
12            fmt.Println("Input tidak valid, masukkan antara [Y/y or N/n]!")
13        }
14    }
15 }

```

Fungsi validasi disini berguna untuk menahan user sebelum sistem benar - benar mengeksekusi program utama yang ada di setiap menu. Fungsi ini bekerja dengan cara meminta user untuk menginput nilai string [Y/N] kepada sistem untuk melanjutkan eksekusi program ataupun melakukan cancel pada programnya. Dibuat didalam loop repeat-until, sehingga fungsi ini akan selesai dieksekusi apabila user memasukan nilai yang tepat, jika tidak fungsi ini akan terus menerus meminta input string hingga nilainya sesuai dengan flow di percabangan.

3. Fungsi Helper (cekInputKosong)



```
1 func cekInputKosong(input string) bool {
2     if input == "" {
3         return false
4     } else {
5         return true
6     }
7 }
```

Berikutnya ada fungsi cekInputKosong, kami membuat fungsi ini untuk mencegah user memasukkan input nilai string kosong pada proses input nama perangkat dan ruangan di fitur CRUD. Kita menggunakan output berupa bool untuk memudahkan validasi di dalam fitur. Jika user memasukkan input string kosong (atau langsung enter), maka fungsi ini akan dieksekusi setelah proses input di fitur dijalankan.

4. Fungsi Helper (Konversi kWh)



```
1 func hitungKonsumsi(perangkat Perangkat) float64 {
2     return (perangkat.dayaw * perangkat.waktu) / 1000
3 }
```

Berikutnya ada fungsi hitungKonsumsi. Kami membuat fungsi ini karena, dalam pengukuran konsumsi listrik pada perangkat, besar kecilnya konsumsi suatu perangkat elektronik itu balik berdasarkan berapa lama penggunaan dari device terkait dan juga

seberapa besar perangkat tersebut memakan daya listrik. Maka dari itu kami membuat fungsi untuk menghitung sekaligus melakukan konversi dari input dayaW (Watt) -> kWh dengan rumus $\text{watt} * \text{waktu} / 1000$. Dengan fungsi ini, pengguna akan lebih mudah melakukan *tracking* pada perangkat elektronik mereka.

5. Fungsi Helper (Tampilkan perangkat)

```
1 func tampilkanPerangkat() {
2     // Melakukan cek apabila isi slice = 0
3     if cekAdaPerangkat() {
4         return
5     }
6
7     fmt.Println("===== DATA PERANGKAT =====")
8
9     for i := 0; i < len(dataPerangkat); i++ {
10        fmt.Println("Data ke-", i+1)
11        fmt.Println("Nama Perangkat :", dataPerangkat[i].namaPerangkat)
12        fmt.Println("Ruangan      :", dataPerangkat[i].ruangan)
13        fmt.Println("Daya (Watt)   :", dataPerangkat[i].dayaW)
14        fmt.Println("Durasi (Jam)    :", dataPerangkat[i].waktu)
15        fmt.Println("Konsumsi      :", hitungKonsumsi(dataPerangkat[i]), "kWh")
16        fmt.Println("-----")
17    }
18 }
```

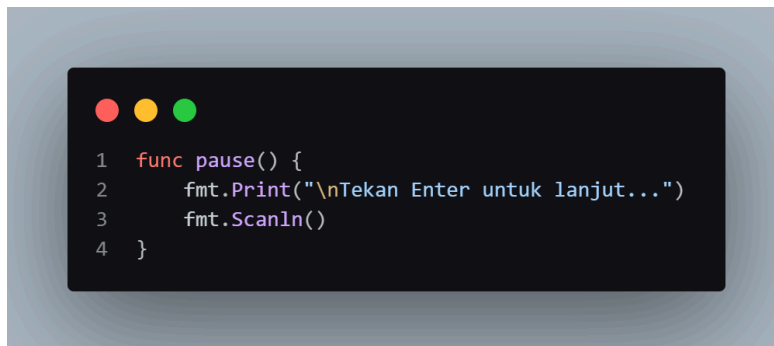
Selanjutnya ada fungsi `tampilkanPerangkat`. Karena semua fitur - fitur aplikasi kami kebanyakan akan menampilkan daftar perangkat yang ada terlebih dahulu, kami membuat fungsi ini untuk membantu proses *development* agar lebih cepat dan efisien. Fungsi ini bekerja dengan cara for loop dari indeks ke-0 hingga panjang array dari `dataPerangkat`.

6. Fungsi Helper (Clear screen)

```
1 func clearScreen() {
2     var cmd *exec.Cmd
3
4     if runtime.GOOS == "windows" { // windows
5         cmd = exec.Command("cmd", "/c", "cls")
6     } else { // mac
7         cmd = exec.Command("clear")
8     }
9
10    // execute command
11    cmd.Stdout = os.Stdout
12    cmd.Run()
13 }
```

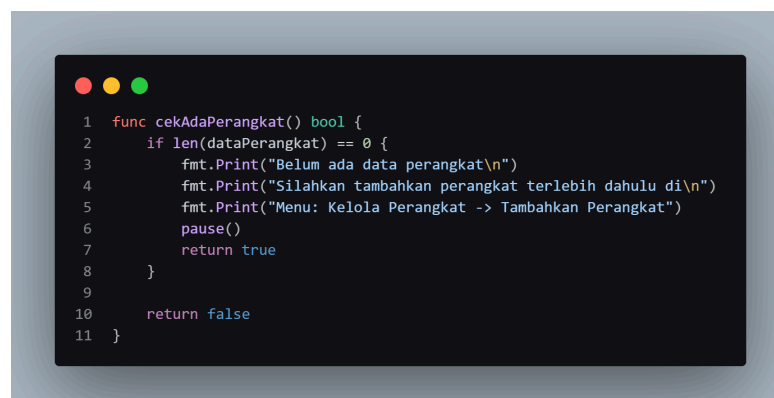
Kemudian ada fungsi `clearScreen`. Karena apps ini berbasis CLI, maka setiap output yang keluar akan memenuhi terminal. Dan tujuan kami membuat fungsi ini untuk membersihkan secara otomatis terminal agar tampilannya lebih bersih. Fungsi ini bekerja dengan cara menggunakan import dari `command execution`, dimana fungsi ini akan melakukan cek dari device pengguna untuk menjalankan command pembersihan terminal. Adanya percabangan di dalam fungsi ini memungkinkan fungsi ini bisa mendeteksi perangkat yang sedang dijalankan adalah windows ataupun mac, sehingga program bisa mengeksekusi command yang tepat berdasarkan sistem operasi yang ada.

7. Fungsi Helper (Pause)



Berikutnya ada fungsi `pause`. Karena program ini berbasis CLI, maka program ini akan terus running perintah hingga tertahan pada program input. Fungsi ini kami bikin sekaligus sebagai *support* dari fungsi `clearScreen`. Karena, tanpa fungsi ini, program akan langsung membersihkan terminal maupun mengeksekusi program selanjutnya setelah melakukan output dan menyebabkan output yang keluar tidak sempat untuk dibaca oleh user karena langsung di bersihkan terminalnya. Fungsi ini bekerja dengan cara menunggu user menekan enter dengan `Scanln` (karena command ini berfungsi untuk menerima nilai enter).

8. Fungsi Helper (cekAdaPerangkat)



Terakhir pada helper yakni fungsi cekAdaPerangkat. Fungsi ini kami buat untuk mencegah user mengakses fitur - fitur apps kami selain fitur menambahkan perangkat jika program kami mendeteksi tidak adanya data didalam slice. Fungsi ini bekerja dengan cara melakukan cek pada slice dataPerangkat dan apabila terdeteksi tidak ada perangkat alias 0, maka fungsi ini akan memberikan nilai true ke sistem yang kami pasang didalam fitur - fitur apps kami dan akan memberikan nilai false apabila terdeteksi ada data didalam slice untuk membuka akses dari fitur - fitur apps yang kami limitasi sebelumnya.

9. Selection Sort (Daya perangkat)

```
1 func selectionSortPerangkat() {
2   n := len(dataPerangkat)
3
4   for i := 0; i < n-1; i++ {
5     maxIdx := i
6
7     for j := i + 1; j < n; j++ {
8       if dataPerangkat[j].dayaW > dataPerangkat[maxIdx].dayaW {
9         maxIdx = j
10      }
11    }
12
13    dataPerangkat[i], dataPerangkat[maxIdx] = dataPerangkat[maxIdx], dataPerangkat[i]
14  }
15 }
```

Selanjutnya ada algoritma sort yakni Selection Sort. Algoritma ini bekerja dengan cara mengambil jumlah data perangkat terlebih dahulu kemudian melakukan perulangan untuk mencari perangkat yang memiliki nilai daya terbesar pada proses pencarian, dan variabel maxidx digunakan untuk menyimpan posisi perangkat dengan daya terbesar yang ditemukan sementara kemudian melakukan perulangan ke 2 dengan membandingkan nilai daya watt setiap perangkat dengan nilai daya yang lebih besar dan kemudian nilai dari maxidx diperbarui.

10. Insertion Sort (Abjad perangkat)

```
1 func insertionSortPerangkat() {
2   n := len(dataPerangkat)
3
4   for i := 1; i < n; i++ {
5     key := dataPerangkat[i]
6     j := i - 1
7
8     for j >= 0 && dataPerangkat[j].namaPerangkat > key.namaPerangkat {
9       dataPerangkat[j+1] = dataPerangkat[j]
10      j = j - 1
11    }
12    dataPerangkat[j+1] = key
13  }
14 }
```


Selanjutnya ada algoritma sort selanjutnya yaitu insertionSort yang digunakan untuk mengurutkan data perangkat berdasarkan nama perangkat secara alfabet (A-Z). Algoritma Insertion Sort bekerja dengan mengambil satu data kemudian membandingkannya dengan data sebelumnya. Jika urutannya belum sesuai, data akan digeser hingga ditemukan posisi yang tepat. Proses ini dilakukan secara berulang sampai seluruh data perangkat terurut berdasarkan abjad.

11. Selection Sort (Konsumsi daya tertinggi perangkat)

A screenshot of a code editor with a dark background and light-colored text. The code is written in Go and implements a Selection Sort algorithm. It starts with a function signature 'func sortKonsumsiTertinggi()' followed by a series of lines that calculate the length of 'dataPerangkat', iterate through the array to find the maximum value, and then swap it with the element at the current index. The code is numbered from 1 to 18.

```
1 func sortKonsumsiTertinggi() {
2
3     n := len(dataPerangkat)
4
5     for i := 0; i < n-1; i++ {
6
7         maxIdx := i
8
9         for j := i + 1; j < n; j++ {
10
11             if hitungKonsumsi(dataPerangkat[j]) > hitungKonsumsi(dataPerangkat[maxIdx]) {
12                 maxIdx = j
13             }
14         }
15
16         dataPerangkat[i], dataPerangkat[maxIdx] = dataPerangkat[maxIdx], dataPerangkat[i]
17     }
18 }
```

Selanjutnya kami masih menggunakan fungsi selection sort untuk mengurutkan konsumsi daya pada perangkat dari yang tertinggi ke yang terendah, dan cara kerja fungsi ini sebenarnya cukup sama dengan daya perangkat akan tetapi pada fungsi kali ini fungsi ini mencari perangkat dengan konsumsi tertinggi dan menempatkannya di depan, dan terus dilakukan sampai data perangkat terurut berdasarkan konsumsi listrik.

12. Binary Search (Mencari berdasarkan nama ruangan)

```
1 func cariRuangan() {
2
3     var cariberduangan string
4
5     clearScreen()
6
7     fmt.Print("Masukkan nama ruangan yang dicari : ")
8     fmt.Scanln(&cariberduangan)
9
10    if !cekInputKosong(cariberduangan) {
11        fmt.Print("Input tidak boleh kosong!")
12        pause()
13        return
14    }
15
16    sortRuangan()
17
18    // Start batas kiri sama batas kanan
19    kiri := 0
20    kanan := len(dataPerangkat) - 1
21    ketemu := -1
22
23    for kiri <= kanan {
24
25        tengah := (kiri + kanan) / 2
26
27        if dataPerangkat[tengah].ruangan == cariberduangan {
28            ketemu = tengah
29            break
30        } else if dataPerangkat[tengah].ruangan < cariberduangan {
31            kiri = tengah + 1
32        } else {
33            kanan = tengah - 1
34        }
35    }
36
37    if ketemu == -1 {
38        fmt.Println("\nRuangan tidak ditemukan!")
39        pause()
40        return
41    }
42
43    fmt.Println("\n===== HASIL PENCARIAN =====")
44
45    for i := 0; i < len(dataPerangkat); i++ {
46
47        if dataPerangkat[i].ruangan == cariberduangan {
48
49            fmt.Println("-----")
50            fmt.Println("Nama Perangkat :", dataPerangkat[i].namaPerangkat)
51            fmt.Println("Ruangan      :", dataPerangkat[i].ruangan)
52            fmt.Println("Daya (Watt)   :", dataPerangkat[i].dayaW)
53            fmt.Println("Durasi (Jam)  :", dataPerangkat[i].waktu)
54        }
55    }
56
57    pause()
58
59 }
```

Pada kali ini kita menggunakan Binary Search untuk mencari perangkat berdasarkan nama ruangan yang dimasukkan oleh user, sebelum proses pencarian data diurutkan terlebih dahulu menggunakan fungsi `sortRuangan()` agar dapat menggunakan algoritma Binary Search. Binary Search bekerja dengan menentukan batas kiri dan batas kanan data, kemudian memeriksa data yang berada di posisi tengah. Jika data tengah sama dengan ruangan yang dicari maka pencarian selesai. Jika ruangan yang dicari lebih besar dari data tengah, pencarian dilanjutkan ke bagian kanan. Sebaliknya, jika lebih kecil, pencarian dilanjutkan ke bagian kiri. Proses ini terus dilakukan hingga data ditemukan atau seluruh kemungkinan pencarian habis. Apabila ruangan ditemukan, program akan menampilkan seluruh perangkat yang berada pada ruangan tersebut. Namun jika tidak ditemukan, program akan menampilkan pesan bahwa ruangan tidak tersedia pada data.

13. Sequential Search (Mencari berdasarkan nama perangkat)

```
1 func cariNamaPerangkat() {
2
3     var cariberdnama string
4     var ketemu bool
5
6     clearScreen()
7
8     fmt.Print("Masukkan nama perangkat yang dicari : ")
9     fmt.Scanln(&cariberdnama)
10
11     if !cekInputKosong(cariberdnama) {
12         fmt.Print("Input tidak boleh kosong!")
13         pause()
14         return
15     }
16
17     fmt.Println("\n===== HASIL PENCARIAN =====")
18
19     // Proses Looping buat nyari perangkat sesuai inputan "cariberdnama"
20     for i := 0; i < len(dataPerangkat); i++ {
21
22         // Membandingkan namaPerangkat di slice dengan inputan user
23         if dataPerangkat[i].namaPerangkat == cariberdnama {
24
25             fmt.Println("Nama Perangkat :", dataPerangkat[i].namaPerangkat)
26             fmt.Println("Ruangan      :", dataPerangkat[i].ruangan)
27             fmt.Println("Daya (Watt)   :", dataPerangkat[i].dayaW)
28             fmt.Println("Durasi (Jam)   :", dataPerangkat[i].waktu)
29             fmt.Println("-----")
30
31             ketemu = true // Memberikan indikator karena data sudah ketemu
32         }
33     }
34
35     // Kondisi bool default = false
36     if !ketemu {
37         fmt.Println("Perangkat tidak ditemukan!")
38     }
39
40     pause()
41 }
```

Pada kali ini kita menggunakan sequential search untuk mencari perangkat berdasarkan nama perangkat itu sendiri. Fungsi ini menerapkan algoritma sequential search yaitu melakukan pencarian secara berurutan dari data pertama hingga akhir pada slice data perangkat. Proses pencarian dilakukan menggunakan perulangan (for) yang memeriksa setiap data satu per satu. Pada setiap iterasi, program membandingkan nama perangkat yang tersimpan dengan nama yang diinputkan oleh user. Jika ditemukan data yang sesuai, maka informasi perangkat akan ditampilkan dan variabel (ketemu) akan diubah menjadi (true) sebagai penanda bahwa data berhasil ditemukan. setelah semuanya diperiksa dan masih tetap (false) maka tidak ada perangkat yang sesuai dengan input user sehingga program menampilkan bahwa data tidak ditemukan/tidak sesuai.

BAB IV

TANTANGAN DAN SOLUSI

Tentunya dalam proses pembuatan dan pengembangan aplikasi ini, kami menemukan banyak kerumitan dan juga bug dari. Berikut beberapa kendala yang kami hadapi dan juga solusi yang kami terapkan.

1. Validasi input di sistem pilih menu

Kendala kami disini adalah, kami menggunakan switch-case untuk pemilihan menu nya. Namun, kendala kami sebelumnya adalah di pengecekan validasi input. Kami memikirkan apabila user memasukkan nilai yang tidak seharusnya (seperti memasukkan string walaupun program meminta angka) itu bisa berakibat pada bug di apps kami. Solusi dari kami adalah dengan menginisiasi variabel “pilih” pada menu sebagai string. Karena fungsi pilihmenu hanya perlu membaca input yang pasti dari user (yakni berupa angka 1 2 dan seterusnya), maka menggunakan string dengan inisiasi didalam case seperti (“1”, “2”, “3”) sudah sangat cukup untuk kami dalam melakukan validasi input dari user.

2. Bug saat user menekan “Enter” saat memasuki sub menu

Bug lainnya kami temukan saat kami melakukan test pada apps kami, yakni kami menyadari sesuatu dimana jika user sedang berada pada main menu aplikasi lalu memilih submenu, lalu user tersebut kembali ke main menu, saat user menekan enter maka program langsung memasuki submenu yang sebelumnya diakses, bukan tertahan di main menu. Setelah kami melakukan *debugging*, kami mendapatkan solusi untuk mengatasi hal ini dengan cara memberikan variabel pilih = “” diawal loop sebelum proses input dimulai. Hal ini kami lakukan apabila user menekan enter saja walaupun tidak mengisi apa - apa, maka program akan memasuki kondisi default di dalam switch-case.

3. Duplikasi data saat proses input dan mengubah data

Masalah ini muncul saat kami sedang melakukan responsi dengan asisten praktek kami, sebenarnya, secara jalan program dan lain - lainnya itu tidak ada crash ataupun masalah. Namun, dari kami memikirkan dari sisi *database* (dimana duplikasi data itu adalah masalah yang serius). Contoh: didalam slice sudah ada AC di ruangan Kamar_Tidur, namun saat user menambahkan perangkat dengan nama dan ruangan yang sama persis, maka program tersebut tetap berjalan dan juga tetap menyimpan data tersebut. Walaupun setelah kami test dengan fitur sort maupun search tidak ada masalah, namun kami memikirkan

bahwasannya ada duplikasi data didalam sistem kami yang berpotensi menjadi masalah serius apabila aplikasi ini ingin dikembangkan lebih luas lagi. Dari kami memberikan solusi dengan cara, saat user selesai melakukan input namaPerangkat dan ruangan pada menu tambahPerangkat maupun ubahPerangkat, maka kami menambahkan for loop yang dimulai dari indeks 0 hingga panjang slice data yang didalamnya ada percabangan untuk memeriksa apabila inputan namaPerangkat dari user itu sama dengan namaPerangkat yang ada di slice dan inputan ruangan dari user itu sama dengan ruangan yang ada di slice, maka kami mencetak pemberitahuan jika nama perangkat sudah ada diruangan yang sama lalu kami return untuk keluar dari menu terkait. Walaupun setelah kami tes tanpa fungsi ini apps tetap berjalan, namun kami tetap menambahkan validasi ini untuk mencegah adanya duplikasi data kedepannya.

4. Proses input untuk dayaW

Masalah ini kami temukan saat melakukan proses input daya watt dari perangkat, dimana jika user memasukkan nilai 0 atau minus, program tetap berjalan. Solusi dari kami adalah menambahkan proses validasi setelah proses input angka watt dimana jika angka yang dimasukkan itu ≤ 0 , maka program akan kami return untuk mencegah user menginput nilai yang salah.

5. Proses membuat algoritma Sorting dan Searching

Ini merupakan hal yang cukup rumit saat proses ini. Dimana, kami sempat kesulitan dalam merancang proses sorting maupun searching ini karena kami harus menggunakan semua pola yang tertera pada panduan tugas besar dan cukup kompleks. Solusi dari kami yakni kami membaca dan mencari sumber dari algoritma terkait mengenai cara kerja dan cara menulisnya, karena secara garis besar kami menemukan bahwa cara penulisan algoritma sorting dan searching yang digunakan di aplikasi ini perbedaannya hanya terletak pada variabel yang dibandingkan maupun *ascending* dan *descending*, selebihnya struktur utama algoritma ini sama.

BAB V

KESIMPULAN DAN REKOMENDASI

5.1 Kesimpulan

Pada project tugas besar ini, kami telah berhasil membangun aplikasi **PowerLog**, yaitu sebuah sistem berbasis CLI yang berfungsi untuk mencatat, mengelola, dan melakukan analisis konsumsi daya listrik pada suatu perangkat elektronik. Program ini kami kembangkan dengan mengikuti standarisasi, referensi, dan spesifikasi yang diberikan oleh dosen pengampu melalui panduan pengerjaan tugas besar yang diberikan kepada kami. Meskipun menghadapi berbagai tantangan teknis dalam proses pengembangannya, berkat dukungan dari asisten praktikum serta kerja satu tim yang solid, project mata kuliah Algoritma Pemrograman ini dapat kami selesaikan tepat waktu.

Pengembangan aplikasi ini tidak hanya berfokus pada penerapan CRUD (Create, Read, Update, Delete), tetapi juga telah berhasil mengimplementasikan struktur data struct dan array/slice, serta pengaplikasian algoritma kompleks. Penerapan algoritma Sorting (Selection Sort dan Insertion Sort) serta Searching (Binary Search dan Linear Search) di dalam sistem terbukti berjalan dengan efisien. Secara keseluruhan, aplikasi PowerLog kami menunjukkan stabilitas yang baik, dan mampu menangani validasi input untuk mencegah error, serta memiliki proteksi terhadap duplikasi data, yang dimana pengalaman ini dapat menambah wawasan kami dalam merancang suatu sistem maupun kemampuan problem solving kedepannya.

5.2 Rekomendasi

Saran lanjutan untuk pengembangan aplikasi ini kami ini antara lain:

1. Memperbaiki tampilan CLI dari aplikasi ini agar lebih mudah untuk dibaca
2. Memberikan fitur tracking konsumsi listrik tiap bulan nya
3. Pengembangan aplikasi berbasis GUI ataupun web (bukan lagi berbasis CLI yang hanya bisa dijalankan di device laptop maupun pc)
4. Menambahkan fitur - fitur yang lebih menarik dan berguna untuk aplikasi ini.

TAUTAN GITHUB

Link Repository: <https://github.com/yogidharmaa/powerlog-apps-tubes>

REFERENSI

<https://pkg.go.dev/os>

<https://pkg.go.dev/runtime>

<https://pkg.go.dev/os/exec>

<https://pkg.go.dev/fmt>

<https://dev.to/adnanbabakan/searching-algorithms-in-go-cop>

https://go.dev/doc/effective_go#blank

Sumber Materi dari Praktikum Algoritma & Pemrograman 2